

PX4 Autosoaring — Complete Annotated Diff

Every changed line explained:

Pictures/PX4-Autopilot (modified) vs Music/PX4-Autopilot (original)

Radhouene — Fixed-Wing Autosoaring Project

May 2026

Contents

1	Overview of all changed files	3
2	msg/CMakeLists.txt	3
3	msg/AutosoaringControl.msg (New file)	3
4	msg/AutosoaringStatus.msg (New file)	4
5	src/modules/uxrce_dds_client/dds_topics.yaml	5
6	src/modules/commander/Commander.cpp	6
6.1	Hunk 1 — new mode <code>soar:*</code> subcommands (lines +417–450)	6
6.2	Hunk 2 — DO_REPOSITION param2 fix (lines +782–95)	7
7	src/modules/fw_pos_control/fw_path_navigation_params.c	8
8	src/lib/tecs/TECS.hpp	9
8.1	Hunk 1 — new fields in <code>struct Param</code>	9
8.2	Hunk 2 — new flag in <code>struct Flag</code>	9
8.3	Hunk 3 — <code>_calcThrottleControlSteRate</code> signature change	10
8.4	Hunk 4 — new state variable	10
8.5	Hunk 5 — public soaring interface	10
8.6	Hunk 6 — default initialisation	10
9	src/lib/tecs/TECS.cpp	10
9.1	Mod 1 — pass flag to <code>_calcThrottleControlSteRate</code>	11
9.2	Mod 2 — acceleration energy cap in <code>_calculateTASSetpointRateLimit</code>	11
9.3	Mod 3 — underspeed mitigation bypass	11
9.4	Mod 4 — energy weighting override	11
9.5	Mod 5 — pitch integrator partial reset on transition	12
9.6	Mod 6 — throttle integrator decay	12
9.7	Mod 7 — throttle output hard-zero	12
10	src/modules/navigator/navigator_main.cpp	13
10.1	Hunk 1 — DO_REPOSITION param3 loiter geometry	13

10.2	Hunk 2 — NAVIGATION_STATE_ORBIT mapping	13
11	src/modules/navigator/mission.cpp	14
12	src/modules/navigator/mission_base.cpp	14
12.1	Hunk 1 — sequence guard in <code>onMissionUpdate()</code>	14
12.2	Hunk 2 — clear guard after new mission items	15
12.3	Hunk 3 — capture sequence at deactivation (<code>on_inactivation()</code>)	15
12.4	Hunk 4 — clear guard and reset triplets at re-activation	15
12.5	Hunk 5 — clear guard at mission loop	15
12.6	Hunk 6 — clear guard on new mission upload	15
13	src/modules/navigator/mission_base.h	16
14	ROMFS/px4fmu_common/init.d-posix/airframes/4008_gz_advanced_plane	16
15	Complete data-flow summary	16

1 Overview of all changed files

File	What changed
msg/CMakeLists.txt	+2 lines: register the two new message files
msg/AutoSoaringControl.msg	New file (44 lines): companion→FMU command
msg/AutoSoaringStatus.msg	New file (22 lines): FMU→companion feedback
dds_topics.yaml	+6 lines: bridge both topics via XRCE-DDS
Commander.cpp	+31 added / -1 removed: soar: CLI subcommands + DO_REPOSITION fix
FixedwingPositionControl.hpp	+52 added: includes, subscribers, publishers, state variables, parameters
fw_path_navigation_params.c	+154 added: 9 new soaring parameters with full QGC metadata
TECS.hpp	+22 added: gliding flag, params, setters, integrator state
TECS.cpp	+105 added / -8 removed: 7 gliding-mode modifications
navigator_main.cpp	+22 added / -7 removed: param3 loiter geometry + ORBIT state
mission.cpp	+1 added: clear paused-seq on index jump
mission_base.cpp	+18 added / -1 moved: mission-pause seq guard in 5 locations
mission_base.h	+1 added: <code>_mission_paused_seq</code> member
4008_gz_advanced_plane	-1/+1: FW_THR_MIN 0.05→0.00

2 msg/CMakeLists.txt

Role of this file: PX4's CMake build system reads `set(msg_files ...)` in this file and runs the uORB code generator on every listed `.msg` file. Without an entry here the C struct header (e.g. `autoSoaringControl_s`) is never generated and the module will not compile.

```
@@ -40,6 +40,8 @@
    AckermannVelocitySetpoint.msg
    ActionRequest.msg
    ActuatorArmed.msg
+   AutoSoaringControl.msg      <-- NEW: companion sends soaring mode to FMU
+   AutoSoaringStatus.msg      <-- NEW: FMU sends phase feedback to companion
    ActuatorControlsStatus.msg
```

Line +1 AutoSoaringControl.msg: Tells the generator to produce `autoSoaringControl_s` and `ORB_ID(autoSoaringControl)`. The `uxrce_dds_client` will subscribe to it and forward DDS data from the companion into this uORB topic.

Line +2 AutoSoaringStatus.msg: Tells the generator to produce `autoSoaringStatus_s` and `ORB_ID(autoSoaringStatus)`. The `uxrce_dds_client` will read from this topic and publish it on the DDS bus so the companion can subscribe.

3 msg/AutoSoaringControl.msg (New file)

Direction: companion (ROS 2) → FMU via XRCE-DDS `/fmu/in/autoSoaringControl`.
Consumer: `FixedwingPositionControl` reads it in `autoSoaringControl_poll()`.
Design rationale: replaces two booleans (`glide_mode_enabled`, `thermal_mode_enabled`) with a single `uint8` integer. The old design had an ambiguous `glide=true`, `thermal=true` state and a hidden dimension in `FW_GLIDE_MODE`.

```
+uint64 timestamp
```

Mandatory PX4 field. The companion sets it to 0; the DDS bridge and uORB machinery do not use it for control, but it is stored in ULog for time-alignment during post-flight analysis.

```
+uint8 soaring_mode
+uint8 SOARING_OFF           = 0 # Normal powered flight
+uint8 SOARING_GLIDE_FIXED   = 1 # Engine-off, fixed EAS = FW_GLIDE_AIRSPD
+uint8 SOARING_GLIDE_POLAR   = 2 # Engine-off, polar best-glide EAS = sqrt(B/A)
+uint8 SOARING_THERMAL_LOITER = 3 # Thermalling, radius-guided loiter via navigator
+uint8 SOARING_THERMAL_BANK  = 4 # Thermalling, direct bank-angle (companion controls)
```

The single field that drives all FMU soaring logic. **0** returns to powered flight. **1** enables glide at a fixed user-specified EAS. **2** computes EAS from the aircraft polar ($\sqrt{B/A}$). **3** tells the navigator to loiter at a companion-supplied centre. **4** lets the companion dictate the bank angle directly (for centering algorithms).

```
+float32 loiter_radius_m # NaN = auto-derive from bank_angle_cmd
+float32 loiter_lat      # Thermal centre latitude [deg]; NaN or 0 = current pos
+float32 loiter_lon      # Thermal centre longitude [deg]; NaN or 0 = current pos
```

Used only when `soaring_mode = 3` (THERMAL_LOITER). The FMU forwards these to the navigator via `VEHICLE_CMD_DO_REPOSITION` with the signed radius packed into `param3`. NaN sentinels avoid a separate boolean for “use default”.

```
+float32 airspeed_cmd # MacCready speed [m/s EAS]; mode 1 only. NaN=FW_GLIDE_AIRSPD
+float32 bank_angle_cmd # Bank angle magnitude [deg]; modes 3 and 4
+bool loiter_clockwise # true = CW (right), false = CCW (left). Default: true
```

`airspeed_cmd` overrides `FW_GLIDE_AIRSPD` for mode 1; the companion can implement a MacCready speed-to-fly calculator. `bank_angle_cmd` sets the loiter tightness; the FMU converts it to a minimum radius. `loiter_clockwise` controls turn direction — useful for thermal asymmetries.

```
+uint8 soaring_phase # Companion-reported phase (metadata only, FMU does not act)
+uint8 SOARING_PHASE_CRUISE = 0
+uint8 SOARING_PHASE_GLIDING = 1
+uint8 SOARING_PHASE_SEARCHING = 2
+uint8 SOARING_PHASE_CENTERING = 3
+uint8 SOARING_PHASE_THERMAL = 4
```

The companion writes its internal state-machine phase here so it is recorded in ULog. The FMU reads but ignores it for control. Note: the *FMU-derived* phase (what the FMU is actually doing) lives in `AutoSoaringStatus.fmu_soaring_phase`.

4 msg/AutoSoaringStatus.msg (New file)

Direction: FMU → companion via XRCE-DDS `/fmu/out/autoSoaringStatus`.
Publisher: `FixedwingPositionControl::publish_autoSoaringStatus()`.

Publication cadence: 2 Hz during active soaring (SOURCE_PERIODIC); 1 Hz for 5 s after `soar:off`; one-shot on watchdog event.

```
+uint64 timestamp
```

Set to `hrt_absolute_time()` at publish time. Allows the companion to compute message age and detect missed samples.

```
+uint8 source
+uint8 SOURCE_CLI_SOARING_OFF      = 0 # Operator soar:off via VEHICLE_CMD_CUSTOM_0
+uint8 SOURCE_STALENESS_WATCHDOG   = 1 # Companion silent >2 s, soaring forced OFF
+uint8 SOURCE_PERIODIC             = 2 # 2 Hz heartbeat during active soaring
```

`source` tells the companion *why* this message was published.

0 (CLI_SOARING_OFF): The ground operator issued commander mode `soar:off`. The companion must stop asserting soaring on for at least `dds_inhibit_until_us` - now microseconds.

1 (STALENESS_WATCHDOG): The companion has been silent for more than 2 s. The FMU disabled soaring autonomously. The companion should check its DDS link.

2 (PERIODIC): Normal 2 Hz heartbeat carrying the live `fmu_soaring_phase`.

```
+uint8 soaring_mode_effective # Mode in _autosoaring_control after the event
```

The value the FMU's internal `_autosoaring_control.soaring_mode` was set to when the message was published. After `soar:off` or the watchdog this is always 0. During SOURCE_PERIODIC it reflects the current commanded mode (1-4).

```
+uint64 dds_inhibit_until_us # Absolute HRT time [us]; 0 = no inhibit
```

Non-zero only while a `soar:off` 5-second inhibit is active. The companion computes the remaining window as `dds_inhibit_until_us - current_time_us`. After this timestamp any DDS publish of `soaring_mode` `!= 0` will be accepted again.

```
+uint8 fmu_soaring_phase
+uint8 SOARING_PHASE_CRUISE      = 0 # Powered cruise (soaring OFF or forbidden)
+uint8 SOARING_PHASE_GLIDING     = 1 # Engine-off glide along mission
+uint8 SOARING_PHASE_SEARCHING   = 2 # Thermal mode ON, loiter centre not yet set
+uint8 SOARING_PHASE_THERMAL     = 3 # Thermal loiter, centre established
```

The FMU-derived phase, computed from the internal booleans `effective_glide` and `effective_thermal`, and from whether `_soaring_last_lat` is finite (loiter centre known). This is independent of what the companion *commanded*; it tells the companion what is *actually happening* on the FMU side, including implicit mode changes driven by altitude limits.

5 src/modules/uxrce_dds_client/dds_topics.yaml

Role: The `uxrce_dds_client` module bridges uORB topics to ROS 2 DDS. It reads this YAML to decide which topics to publish (FMU→ROS 2) and which to subscribe (ROS 2→FMU). Without an entry here a topic is invisible to the companion even if it exists in uORB.

```
@@ -74,6 +74,9 @@
- topic: /fmu/out/airspeed_validated
  type: px4_msgs::msg::AirspeedValidated

+ - topic: /fmu/out/autosoaring_status      <-- FMU publishes, companion subscribes
+   type: px4_msgs::msg::AutosoaringStatus

- topic: /fmu/out/vtol_vehicle_status
```

Registers the **publications** (FMU→ROS 2) side. The bridge reads ORB_ID(`autosoaring_status`) from uORB and re-publishes it on the DDS bus as `/fmu/out/autosoaring_status` with type `px4_msgs::msg::AutosoaringStatus`. The companion echoes it with `ros2 topic echo /fmu/out/autosoaring_status`.

```
@@ -130,6 +133,9 @@
- topic: /fmu/in/trajectory_setpoint
  type: px4_msgs::msg::TrajectorySetpoint

+ - topic: /fmu/in/autosoaring_control      <-- companion publishes, FMU subscribes
+   type: px4_msgs::msg::AutosoaringControl

- topic: /fmu/in/vehicle_attitude_setpoint
```

Registers the **subscriptions** (ROS 2→FMU) side. The bridge listens on DDS for messages on `/fmu/in/autosoaring_control` and writes them into ORB_ID(`autosoaring_control`) uORB, where `fw_pos_control` picks them up via `_autosoaring_control_sub.update()`.

6 src/modules/commander/Commander.cpp

Role of Commander in PX4: Commander manages the vehicle's high-level state machine: arming, mode transitions, preflight checks, and failsafe responses. It exposes a CLI via the NuttShell, and sends MAVLink commands (VEHICLE_CMD_DO_SET_MODE, VEHICLE_CMD_CUSTOM_0) to other modules.

Why two commands per soaring transition: VEHICLE_CMD_CUSTOM_0(param1=mode) is consumed by `fw_pos_control` to set its internal soaring mode and arm/disarm the DDS inhibit. VEHICLE_CMD_DO_SET_MODE switches the navigator state (AUTO_MISSION or AUTO_LOITER) so the flight path follows the correct behaviour.

6.1 Hunk 1 — new mode `soar:*` subcommands (lines +417-450)

```
@@ -417,6 +417,37 @@
        send_vehicle_command(VEHICLE_CMD_DO_SET_MODE, 1,
                              PX4_CUSTOM_MAIN_MODE_AUTO,
                              PX4_CUSTOM_SUB_MODE_EXTERNAL1);

+        } else if (!strcmp(argv[1], "soar:glide")) {
```

`argv[1]` is the string typed after `commander mode` in the NSH shell or QGC MAVLink Console. So the full command is `commander mode soar:glide`. The check lives inside the existing `if (strcmp(argv[0], "mode"))` block.

```
+        // SOARING_GLIDE_FIXED (mode 1): engine-off, fixed EAS =
+        FW_GLIDE_AIRSPD      send_vehicle_command(vehicle_command_s::VEHICLE_CMD_CUSTOM_0, 1.0f,
+        0.0f);
```

VEHICLE_CMD_CUSTOM_0 is intercepted by `FixedwingPositionControl::vehicle_command_poll()`. `param1 = 1.0` maps to SOARING_GLIDE_FIXED. `param2 = 0.0` is unused for enable commands.

```
+        send_vehicle_command(vehicle_command_s::VEHICLE_CMD_DO_SET_MODE, 1,
+        PX4_CUSTOM_MAIN_MODE_AUTO,
+        PX4_CUSTOM_SUB_MODE_AUTO_MISSION);
```

Switches the navigator to AUTO_MISSION so the aircraft continues following the loaded mission waypoints while gliding.

```
+         } else if (!strcmp(argv[1], "soar:polar")) {
+             // SOARING_GLIDE_POLAR (mode 2): engine-off, polar best-glide EAS =
+             sqrt(B/A)
+             send_vehicle_command(vehicle_command_s::VEHICLE_CMD_CUSTOM_0, 2.0f,
+             0.0f);
+             send_vehicle_command(vehicle_command_s::VEHICLE_CMD_DO_SET_MODE, 1,
+             PX4_CUSTOM_MAIN_MODE_AUTO,
+             PX4_CUSTOM_SUB_MODE_AUTO_MISSION);
```

Same as glide-fixed but `param1 = 2. fw_pos_control` will compute EAS from `sqrt(FW_POLAR_B / FW_POLAR_A)` instead of using `FW_GLIDE_AIRSPD`. Navigator state is the same (AUTO_MISSION).

```
+         } else if (!strcmp(argv[1], "soar:thermal")) {
+             // SOARING_THERMAL_LOITER (mode 3): radius-guided loiter via
+             navigator
+             send_vehicle_command(vehicle_command_s::VEHICLE_CMD_CUSTOM_0, 3.0f,
+             0.0f);
+             send_vehicle_command(vehicle_command_s::VEHICLE_CMD_DO_SET_MODE, 1,
+             PX4_CUSTOM_MAIN_MODE_AUTO,
+             PX4_CUSTOM_SUB_MODE_AUTO_LOITER);
```

`param1 = 3`: thermal loiter. Navigator is switched to AUTO_LOITER so the aircraft circles. The loiter centre is supplied by the companion via subsequent DO_REPOSITION commands once a thermal is found.

```
+         } else if (!strcmp(argv[1], "soar:bank")) {
+             // SOARING_THERMAL_BANK (mode 4): direct bank-angle loiter,
+             companion controls centering
+             send_vehicle_command(vehicle_command_s::VEHICLE_CMD_CUSTOM_0, 4.0f,
+             0.0f);
+             send_vehicle_command(vehicle_command_s::VEHICLE_CMD_DO_SET_MODE, 1,
+             PX4_CUSTOM_MAIN_MODE_AUTO,
+             PX4_CUSTOM_SUB_MODE_AUTO_LOITER);
```

`param1 = 4`: the companion sends a desired bank angle via `bank_angle_cmd`. The FMU applies it directly as a roll override, giving the companion fine-grained control over the thermal centering spiral.

```
+         } else if (!strcmp(argv[1], "soar:off")) {
+             // SOARING_OFF (mode 0): disable soaring, restore powered flight
+             send_vehicle_command(vehicle_command_s::VEHICLE_CMD_CUSTOM_0, 0.0f,
+             0.0f);
+             send_vehicle_command(vehicle_command_s::VEHICLE_CMD_DO_SET_MODE, 1,
+             PX4_CUSTOM_MAIN_MODE_AUTO,
+             PX4_CUSTOM_SUB_MODE_AUTO_MISSION);
```

`param1 = 0`: `fw_pos_control` receives this, sets `soaring_mode = SOARING_OFF`, arms the 5-second DDS inhibit window, resets `_autosoaring_last_recv_us = 0` (suppresses watchdog), and publishes `AutosoaringStatus` with `SOURCE_CLI_SOARING_OFF`. The second command returns the navigator to AUTO_MISSION.

6.2 Hunk 2 — DO_REPOSITION param2 fix (lines +782–95)

```
@@ -751,7 +782,15 @@
+         const bool mode_switch_not_requested = (change_mode_flags & 1) == 0;
+         const bool unsupported_bits_set = (change_mode_flags & ~1) != 0;
+
+         if (mode_switch_not_requested || unsupported_bits_set) {
+             // Soaring: param2==1 requests a mode switch; param2==0 = position-only
+             update.
+             // Stock logic rejected param2==0 with UNSUPPORTED; we now accept it so that
+             // companion thermal loiter repositions work without a mode switch.
+             if (mode_switch_not_requested) {
```

```
+         // Position-only update: accept so navigator processes the new
+         centre
+         cmd_result = vehicle_command_ack_s::VEHICLE_CMD_RESULT_ACCEPTED;
+
+         } else if (unsupported_bits_set) {
+             answer_command(cmd, vehicle_command_ack_s::
+             VEHICLE_CMD_RESULT_UNSUPPORTED);
```

Original behaviour: if `param2` bit 0 was clear (“no mode switch requested”) Commander returned UNSUPPORTED, and DO_REPOSITION was silently dropped.

Problem: The companion sends DO_REPOSITION with `param2 = 0` to update the thermal centre without changing the flight mode (already in AUTO_LOITER). The old rejection caused the loiter centre to never update.

Fix: When `mode_switch_not_requested` is true, immediately ACCEPTED and let the navigator handle the position update. When unsupported bits are set, still reject. The normal mode-switch path (`param2` bit 0 set) is unchanged.

7 src/modules/fw_pos_control/fw_path_navigation_params.c

Role: PX4 parameter definitions for the `fw_pos_control` module. The macros generate QGC metadata (units, min, max, decimal, group) and register the parameter with the PX4 parameter system. All new parameters belong to the FW Soaring group so they appear together in QGC.

```
+PARAM_DEFINE_FLOAT(FW_ALT_MIN, 100.0f);
```

Soaring altitude floor [m]. Below this altitude the FMU ignores all DDS soaring-enable requests and latches `_soaring_forbidden_latched = true`. The latch is released only when the companion explicitly publishes `soaring_mode = 0`. This prevents the aircraft from entering glide near the ground.

```
+PARAM_DEFINE_FLOAT(FW_ALT_MAX, 500.0f);
```

Thermal ceiling [m]. When altitude exceeds this value during thermal mode (`_alt_max_reached = true`), the FMU forces glide regardless of the companion command. Prevents runaway altitude gain in a strong thermal.

```
+PARAM_DEFINE_FLOAT(FW_ALT_HYST, 20.0f);
```

Ceiling hysteresis [m]. Thermal mode is only re-enabled after the aircraft descends below `FW_ALT_MAX - FW_ALT_HYST`. Without this the aircraft would oscillate rapidly between glide and thermal at the ceiling.

```
+PARAM_DEFINE_FLOAT(FW_GLIDE_AIRSPD, 10.0f);
```

Fixed glide EAS setpoint [m/s]. Used by mode 1 (GLIDE_FIXED) and as the hold speed during thermal modes. Passed to TECS via `set_gliding_airspeed_setpoint()`. Ignored by mode 2 (GLIDE_POLAR), which computes speed from the polar.

```
+PARAM_DEFINE_FLOAT(FW_POLAR_A, 0.003f);
```

Polar coefficient a . Used in the sink-rate model: $\dot{h} = a \cdot V^2 + b$. The best-glide EAS is computed as $\sqrt{b/a}$. Fit from flight-test data at three or more airspeeds.

```
+PARAM_DEFINE_FLOAT(FW_POLAR_B, 0.50f);
```

Polar coefficient b [m/s]. Minimum sink rate (at best-glide speed). Together with a , determines the polar best-glide EAS for mode 2.

```
+PARAM_DEFINE_FLOAT(FW_GLIDE_RAMP_T, 2.0f);
```

Throttle ramp duration on glide exit [s]. When soaring is disabled, throttle linearly ramps from 0 to the TECS demand over this duration. Prevents abrupt propwash disturbance and the pitch transient on engine restart.

```
+PARAM_DEFINE_FLOAT(FW_THERMAL_BANK, 40.0f);
```

Default thermalling bank angle [deg]. Used when the companion does not send `bank_angle_cmd` or sends NaN. Steeper bank $\Rightarrow$ tighter loiter circle $\Rightarrow$ better thermal centering, but more induced drag and higher stall speed.

```
+PARAM_DEFINE_FLOAT(FW_GLIDE_I_DECAY, 10.0f);
```

Throttle integrator decay time constant [s]. During gliding the TECS throttle integrator is not updated but decays as $I(t) = I(0) \cdot e^{-t/\tau}$. This brings the integrator to zero before engine restart so there is no throttle surge. Default 10 s: after one thermal circuit (≈ 30 s) the integrator is at $< 5\%$.

8 src/lib/tecs/TECS.hpp

Role: Declares the `TECSControl` class (inner TECS controller) and the outer `TECS` class (altitude and airspeed reference model). All soaring modifications are additions; no existing interface is removed.

8.1 Hunk 1 — new fields in struct Param

```
@@ -238,6 +238,10 @@
    float fast_descend;
+    float gliding_airspeed_setpoint{10.0f};
```

Pre-computed EAS setpoint for glide modes. Set by `fw_pos_control` before calling `TECSControl::update()` so TECS does not need to know about soaring modes or polar parameters.

```
+    float glide_i_decay{10.0f};
```

Time constant (seconds) for exponential throttle integrator decay during gliding. Mirrors `FW_GLIDE_I_DECAY`; passed through the `Param` struct so the decay is computed inside `_calcThrottleControlUpdate()` without accessing external parameters.

8.2 Hunk 2 — new flag in struct Flag

```
@@ -283,6 +287,7 @@
    struct Flag {
        bool airspeed_enabled;
        bool detect_underspeed_enabled;
+        bool gliding_mode_enabled{false};
```

The central gliding-mode flag. When true, TECS applies seven modifications: energy weighting, underspeed bypass, integrator decay, throttle zeroing, accel energy cap, pitch

integrator partial reset, and altitude reference freeze. All modifications guard on this single flag, making the gliding path easy to audit.

8.3 Hunk 3 — `_calcThrottleControlSteRate` signature change

```
ControlValues _calcThrottleControlSteRate(const STERateLimit &limit,
    const SpecificEnergyRates &specific_energy_rate,
-    const Param &param) const;
+    const Param &param, const Flag &flag) const;
```

The `flag` argument is added so that `_calcThrottleControlSteRate` can apply the gliding-mode acceleration energy cap. Previously it had no knowledge of flight mode. All call sites are updated to pass `flag`.

8.4 Hunk 4 — new state variable

```
+    bool _prev_gliding_mode{false};
```

Latches the gliding mode flag from the previous update cycle. Used in `_calcPitchControl()` to detect entry (`false→true`) and exit (`true→false`) transitions so the pitch integrator partial reset is applied exactly once per transition.

8.5 Hunk 5 — public soaring interface

```
+    void set_gliding_mode_enabled(bool enabled)
+    { _control_flag.gliding_mode_enabled = enabled; }
+    bool get_gliding_mode_enabled() const
+    { return _control_flag.gliding_mode_enabled; }
+    void set_gliding_airspeed_setpoint(float eas)
+    { _control_param.gliding_airspeed_setpoint = eas; }
+    void set_glide_i_decay(float tau)
+    { _control_param.glide_i_decay = math::max(tau, 0.1f); }
```

`fw_pos_control` calls `set_gliding_mode_enabled(true/false)` whenever `effective_glide` || `effective_thermal` changes. The other setters are called once per parameter update cycle to propagate QGC parameter changes into TECS without exposing internal structs.

8.6 Hunk 6 — default initialisation

```
-    .fast_descend = 0.f
+    .fast_descend = 0.f,
+    .gliding_airspeed_setpoint = 10.0f,
+    .glide_i_decay = 10.0f,
+    };
    TECSControl::Flag _control_flag{
        .airspeed_enabled = false,
        .detect_underspeed_enabled = false,
+        .gliding_mode_enabled = false,
+    };
```

Explicit zero/default initialisation in the aggregate initialisers. The comma on `.fast_descend` is required by C++17 aggregate initialisation rules when new fields are appended. `gliding_mode_enabled = false` ensures powered-flight behaviour on startup.

9 src/lib/tecs/TECS.cpp

Seven modifications are made to `TECS.cpp`, all guarded by `flag.gliding_mode_enabled` or `_control_flag.gliding_mode_enabled`. None of the existing powered-flight code paths are removed.

9.1 Mod 1 — pass flag to `_calcThrottleControlSteRate`

```
- ControlValues ste_rate{_calcThrottleControlSteRate(limit, specific_energy_rate,
+ param)};
+ ControlValues ste_rate{_calcThrottleControlSteRate(limit, specific_energy_rate,
+ param, flag)};
```

Two call sites updated (lines 250 and 577). The function now has access to the gliding flag so it can skip the induced-drag load-factor correction and apply the correct energy cap.

9.2 Mod 2 — acceleration energy cap in `_calculateTASSetpointRateLimit`

```
- const float max_tas_rate_sp = (param.fast_descend * 0.5f + 0.5f) * limit.
+ STE_rate_max
- // math::max(input.tas, FLT_EPSILON);
+ // In gliding, no thrust available: energy rate bounded by gravity (!
+ STE_rate_min)
+ // not by powered max climb. Using the powered limit produces spurious pitch
+ transients.
+ const float available_accel_energy_rate = flag.gliding_mode_enabled
+ ? fabsf(limit.STE_rate_min) // gravity-only: bounded by
+ glide sink rate
+ : limit.STE_rate_max; // powered: bounded by max
+ climb rate
+ const float max_tas_rate_sp = (param.fast_descend * 0.5f + 0.5f) *
+ available_accel_energy_rate
+ / math::max(input.tas, FLT_EPSILON);
```

In powered flight the aircraft can convert thrust into kinetic energy rapidly, so `STE_rate_max` (derived from max climb rate) is the correct upper bound. In gliding, thrust is zero; the only energy source for acceleration is gravity (trading altitude for speed). Using `|STE_rate_min|` (the glide polar sink rate) as the cap prevents the controller from demanding kinetic energy gains that are physically impossible, which would cause spurious nose-down pitch commands.

9.3 Mod 3 — underspeed mitigation bypass

```
+ if (flag.gliding_mode_enabled) {
+ _ratio_undersped = 0.0f; // suppress false underspeed detection in glide
+ return;
+ }
```

The glide EAS (`FW_GLIDE_AIRSPD`, e.g. 10 m/s) is deliberately below the cruise trim speed. The underspeed detector compares current speed to a boundary derived from `equivalent_airspeed_trim` (cruise speed), so the glide speed always appears to be “starting to underspeed”. This falsely activates mitigation that raises throttle and forces speed-on-elevator weighting — both are already correctly set by the gliding flag. Clearing `_ratio_undersped` and returning early keeps the control path clean.

9.4 Mod 4 — energy weighting override

```
+ if (flag.gliding_mode_enabled) {
+ weight.spe_weighting = 0.0f; // altitude not controlled by pitch in glide
+ weight.ske_weighting = 2.0f; // full speed control via pitch
```

```
+ return weight;
+ }
```

In powered flight TECS balances altitude (SPE) and speed (SKE) control equally. In gliding, throttle is zero so altitude cannot be recovered by adding thrust; only speed (pitch angle) can be commanded. Forcing $w_{SPE} = 0$, $w_{SKE} = 2$ gives full airspeed authority to the pitch channel, which is the standard glider control law.

9.5 Mod 5 — pitch integrator partial reset on transition

```
+ if (flag.gliding_mode_enabled && !_prev_gliding_mode) {
+ _pitch_integ_state *= 0.5f; // glide entry
+ } else if (!flag.gliding_mode_enabled && _prev_gliding_mode) {
+ _pitch_integ_state *= 0.5f; // glide exit
+ }
+ _prev_gliding_mode = flag.gliding_mode_enabled;
```

Entry (powered → glide): The cruise integrator holds an altitude-control bias. When gliding starts, energy weighting switches to speed-only. The altitude bias now fights the speed setpoint and causes an airspeed overshoot for several seconds. Halving it removes most of the mismatch while retaining CG/trim knowledge.

Exit (glide → powered): The glide integrator has wound up to hold glide speed at zero throttle. When throttle ramps back in, this integrator still commands nose-down pitch, producing a large pitch-rate spike. Halving it symmetrically suppresses the spike.

Partial reset ($\times 0.5$) is a compromise: fast mode handover without fully discarding steady-state trim information.

9.6 Mod 6 — throttle integrator decay

```
+ if (flag.gliding_mode_enabled) {
+ const float decay = math::max(param.glide_i_decay, 0.1f);
+ _throttle_integ_state -= dt * _throttle_integ_state / decay;
+ return;
+ }
```

During gliding the normal throttle integrator update loop is skipped entirely. Instead the integrator decays exponentially: $I(t) = I(0) \cdot e^{-t/\tau}$ (discrete approximation: $I \leftarrow dt \cdot I/\tau$). This brings the integrator to zero before engine restart so there is no throttle surge. The `return` prevents the powered-flight integrator code from running.

9.7 Mod 7 — throttle output hard-zero

```
+ if (flag.gliding_mode_enabled) {
+ return param.throttle_min;
+ }
```

The throttle output function returns `param.throttle_min` (which is 0.0 after the ROMFS change in Section 14) regardless of any TECS demand when gliding. This ensures that even if an integrator residual or a setpoint error causes the controller to compute a non-zero throttle demand, the motor is not commanded. Without this guard, the throttle integrator decay of Mod 6 alone would be insufficient to guarantee zero thrust if the aircraft is descending rapidly.

10 src/modules/navigator/navigator_main.cpp

Role of Navigator: Navigator translates high-level navigation states (AUTO_MISSION, AUTO_LOITER, AUTO_RTL, ...) into position-setpoint triplets consumed by `fw_pos_control`. Two changes are made: (1) the `DO_REPOSITION` handler is extended to accept a signed radius in `param3`, and (2) `NAVIGATION_STATE_ORBIT` is mapped to the existing loiter handler.

10.1 Hunk 1 — DO_REPOSITION param3 loiter geometry

```
@@ -392,17 +392,30 @@
                                rep->current.loiter_pattern =
                                    position_setpoint_s::LOITER_TYPE_ORBIT;
                                }
                                rep->current.loiter_direction_counter_clockwise =
                                curr->current.loiter_direction_counter_clockwise;
                                }
                                rep->previous.timestamp = hrt_absolute_time();
                                rep->current.valid = true;
                                rep->current.timestamp = hrt_absolute_time();
                                rep->next.valid = false;
                                _time_loitering_after_gf_breach = 0;
                                rep->current.loiter_direction_counter_clockwise =
                                curr->current.loiter_direction_counter_clockwise;
                                }
                                rep->previous.timestamp = hrt_absolute_time();
                                rep->current.valid = true;
                                rep->current.timestamp = hrt_absolute_time();
                                rep->next.valid = false;
```

The first part of the change is a **de-indent** — the closing braces were at the wrong indentation level in the original (a pre-existing formatting issue). The logic is identical; only whitespace changes. This is necessary to make the subsequent additions syntactically correct.

```
+ // Honour explicit loiter radius and direction in param3.
+ // Convention: |param3| = radius [m]; sign = direction:
+ // param3 > 0    CW (loiter_direction_counter_clockwise = false)
+ // param3 < 0    CCW (loiter_direction_counter_clockwise = true)
+ // param3 == 0 or NaN use NAV_LOITER_RAD, keep existing
+
+ direction
+     if (PX4_ISFINITE(cmd.param3) && fabsf(cmd.param3) > FLT_EPSILON) {
+         rep->current.loiter_radius = fabsf(cmd.param3);
+         rep->current.loiter_direction_counter_clockwise = (cmd.
+ param3 < 0.f);
+     } else if (!only_alt_change_requested) {
+         rep->current.loiter_radius = get_loiter_radius();
+         // direction keeps default (false = CW)
+     }
+     _time_loitering_after_gf_breach = 0;
```

Why a signed radius: The MAVLink `DO_REPOSITION` command has no dedicated field for loiter radius. `param3` was unused (0 or NaN). By packing both magnitude and direction into the sign of `param3`, both properties can be communicated in a single float. `fw_pos_control` packs the value as: $param3 = r \cdot (clockwise? + 1 : -1)$.

The `else if (!only_alt_change_requested)` branch ensures that altitude-only repositions (vertical updates, e.g. during climb-to-altitude) do not accidentally reset the loiter radius to the NAV default.

10.2 Hunk 2 — NAVIGATION_STATE_ORBIT mapping

```
+ case vehicle_status_s::NAVIGATION_STATE_ORBIT:
+     // Thermal loiter: reuse the loiter navigation mode.
```

```
+ // The companion sets the centre via DO_REPOSITION (handled above);
+ // we map this nav-state to the existing loiter handler.
+ _pos_sp_triplet_published_invalid_once = false;
+ navigation_mode_new = &_loiter;
+ break;
```

11 src/modules/navigator/mission.cpp

```
@@ -122,6 +122,7 @@
                                // User has actively set new index, reset.
                                _inactivation_index = -1;
                                _mission_paused_seq = -1;
                                return true;
```

When the user manually jumps to a specific mission item (`set_current_mission_index()`), the pause-sequence guard must be cleared too. Without this, the guard installed in `mission_base.cpp` would prevent the jump from taking effect if the new index is lower than the paused sequence.

12 src/modules/navigator/mission_base.cpp

Problem being solved: While the vehicle is in `AUTO_LOITER` (thermalling), the ground station continues to publish the `mission` uORB topic at its normal rate. If a stale or repeated publication carries a lower `current_seq` than the one active when thermal mode was entered, the mission module rewinds to an already-completed waypoint. On mission resume the aircraft flies back to that waypoint.

Fix: A new member `_mission_paused_seq` captures the current sequence at deactivation. While the mission mode is inactive, incoming updates with a lower sequence are silently clamped to the saved value.

12.1 Hunk 1 — sequence guard in onMissionUpdate()

```
@@ -95,13 +95,26 @@
                                const bool mission_items_changed = (new_mission.mission_id != _mission.
                                mission_id);
                                const bool mission_data_changed = checkMissionDataChanged(new_mission);

                                if (new_mission.current_seq < 0) { ... }

                                if (!isActive() && (_mission_paused_seq >= 0) && !mission_items_changed
                                && (new_mission.mission_dataman_id == _mission.mission_dataman_id)
                                && (new_mission.current_seq < _mission_paused_seq)) {
                                    new_mission.current_seq = _mission_paused_seq;
                                }

                                const bool mission_data_changed = checkMissionDataChanged(new_mission);
```

The guard has five conditions:

1. `!isActive()` — only applies while mission mode is paused (e.g. in `AUTO_LOITER`).
2. `_mission_paused_seq >= 0` — a valid pause sequence was captured.
3. `!mission_items_changed` — a genuinely new mission plan (different `mission_id`) is always accepted.
4. Same `mission_dataman_id` — ensures we compare sequences within the same

mission data block.

5. `new_mission.current_seq < _mission_paused_seq` — the incoming update would rewind progress.

`checkMissionDataChanged` is moved after the guard so it sees the corrected sequence.

12.2 Hunk 2 — clear guard after new mission items

```
+ _mission_paused_seq = -1; // new mission plan: discard old pause seq
```

12.3 Hunk 3 — capture sequence at deactivation (`on_inactivation()`)

```
@@ -196,6 +210,7 @@
_inactivation_index = _mission.current_seq;
+ _mission_paused_seq = _mission.current_seq;
```

Called when the vehicle leaves `AUTO_MISSION` (e.g. entering `AUTO_LOITER` for thermalling). Saves the current mission sequence number. The guard in Hunk 1 will use this value to clamp any stale incoming updates.

12.4 Hunk 4 — clear guard and reset triplets at re-activation

```
@@ -246,10 +261,22 @@
checkClimbRequired(_mission.current_seq);
+ // Clear the position setpoint triplet inherited from the previous navigation mode
+ // (e.g.\ loiter setpoint from thermal circling) before building the mission triplet
+
+ // Without this, setActiveMissionItems() snapshots the loiter position into
+ // pos_sp_triplet->previous, causing the aircraft to approach the next waypoint
+ // from the thermal centre direction and appear to "go back" to it.
+ _navigator->reset_triplets();

set_mission_items();
_mission_activation_index = _mission.current_seq;
_inactivation_index = -1;
+ _mission_paused_seq = -1;
```

`reset_triplets()`: When the mission module re-activates, `setActiveMissionItems()` copies the current position-setpoint triplet into `pos_sp_triplet.previous`. If that triplet still holds the thermal loiter centre, the fixed-wing guidance draws a track line *from the thermal* to the next waypoint, causing a visible detour. Resetting the triplets forces the guidance to use the current aircraft position as the track start.

`_mission_paused_seq = -1`: Clears the guard so subsequent mission updates are accepted normally after resume.

12.5 Hunk 5 — clear guard at mission loop

```
+ _mission_paused_seq = -1;
```

12.6 Hunk 6 — clear guard on new mission upload

```
+ _mission_paused_seq = -1;
_mission.timestamp = hrt_absolute_time();
_mission.current_seq = 0;
```

13 `src/modules/navigator/mission_base.h`

```
@@ -331,6 +331,7 @@
mission_s _mission;
float _mission_init_climb_altitude_amsl{NAN};
int _inactivation_index{-1};
+ int32_t _mission_paused_seq{-1};
int _mission_activation_index{-1};
```

Declares `_mission_paused_seq` as a member of `MissionBase`. Initialised to `-1` (guard disabled). It is a `int32_t` to match the type of `mission_s::current_seq`. All six use-sites in `mission_base.cpp` and `mission.cpp` reference this member.

14 `ROMFS/px4fmu_common/init.d-posix/airframes/4008_gz`

Role: This shell script is loaded by PX4 on boot when `SYS_AUTOSTART = 4008`. It sets parameter defaults for the Gazebo-Classical advanced-plane model used in SITL tests. It is the standard test vehicle for fixed-wing autosoaring.

```
@@ -33,7 +33,7 @@
param set-default FW_SPOILERS_LND 0.4

-param set-default FW_THR_MIN 0.05
+param set-default FW_THR_MIN 0.00 # 0.00 = true engine-off; 0.05 keeps 5% throttle
param set-default FW_THR_TRIM 0.25
```

Why this is critical: `FW_THR_MIN` is the minimum throttle floor enforced by TECS. With the default value of 0.05 the motor is never fully cut: even with `set_gliding_mode_enabled(true)` TECS clamps the output at 5% thrust. The aircraft behaves like powered flight at very low throttle rather than a true engine-off glide, and sink-rate measurements are corrupted by the residual thrust.

Setting `FW_THR_MIN = 0.00` allows TECS to command exactly zero throttle, enabling genuine gliding. The TECS hard-zero guard in `TECS.cpp` (Mod 7 in Section 8) then returns `param.throttle_min = 0` directly, guaranteeing no thrust.

Safety note: On real hardware, setting `FW_THR_MIN = 0` is correct for a glider or a motor-glider performing soaring trials. For a powered aircraft where thrust loss is an emergency, a small non-zero floor is advisable.

15 Complete data-flow summary

Companion (ROS 2)

FMU (PX4)

```
[1] Enable glide via DDS
ros2 topic pub /fmu/in/autosoaring_control
soaring_mode: 1
```

XRCE-DDS bridge

ORB_ID(autosoaring_control)

```
fw_pos_control::autosoaring_control_poll()
  check _soaring_dds_inhibit_until_us (if inhibit active → DROP)
  check altitude floor FW_ALT_MIN (if too low → latch + DROP)
  _autosoaring_control = soaring_msg → effective_glide = true
```

```
source: 1   soaring_mode_effective: 0
```

```
TECS::set_gliding_mode_enabled(true)
TECS::set_gliding_airspeed_setpoint(EAS)
```

```
7 TECS gliding modifications active
fw_pos_control publishes AutosoaringStatus (SOURCE_PERIODIC, 2 Hz)
```

```
XRCE-DDS bridge
```

```
/fmu/out/autosoaring_status
fmu_soaring_phase: 1 (GLIDING)
```

```
[2] Operator disables soaring via CLI
pxh> commander mode soar:off
```

```
VEHICLE_CMD_CUSTOM_0 (param1=0)
```

```
fw_pos_control::vehicle_command_poll()
  soaring_mode ← SOARING_OFF
  _soaring_dds_inhibit_until_us = now + 5 s
  _autosoaring_last_recv_us = 0 (suppress watchdog)
  publish_autosoaring_status(SOURCE_CLI_SOARING_OFF,
    SOARING_OFF, inhibit_end, SOARING_PHASE_CRUISE)
```

```
1 Hz republish for 5 s (packet-loss robustness)
```

```
/fmu/out/autosoaring_status
source: 0   dds_inhibit_until_us: <nonzero>
```

```
[3] Companion silent > 2 s (staleness watchdog)
fw_pos_control::Run()
  _autosoaring_last_recv_us > 0 && silence > 2 s
  publish_autosoaring_status(SOURCE_STALENESS_WATCHDOG,
    SOARING_OFF, 0, SOARING_PHASE_CRUISE)
```

```
/fmu/out/autosoaring_status
```